UNIVERSIDAD DE LIMA

FACULTAD DE INGENIERÍA
CARRERA DE INGENIERÍA DE SISTEMAS

Recomendación de oportunidades de contratación pública de servicios

Exposición del código y solución implementada

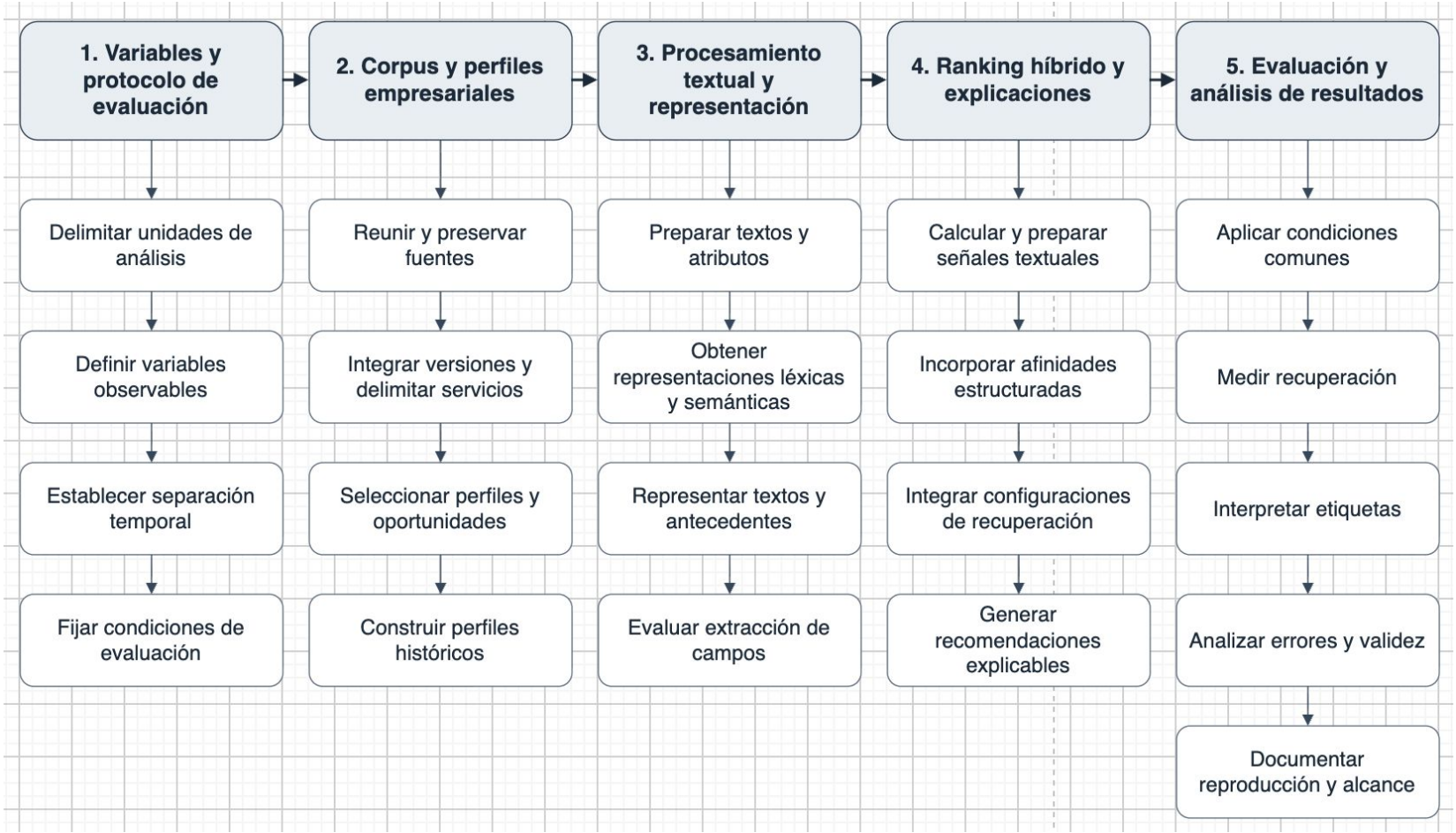
Seminario de Investigación I

13 de octubre de 2026

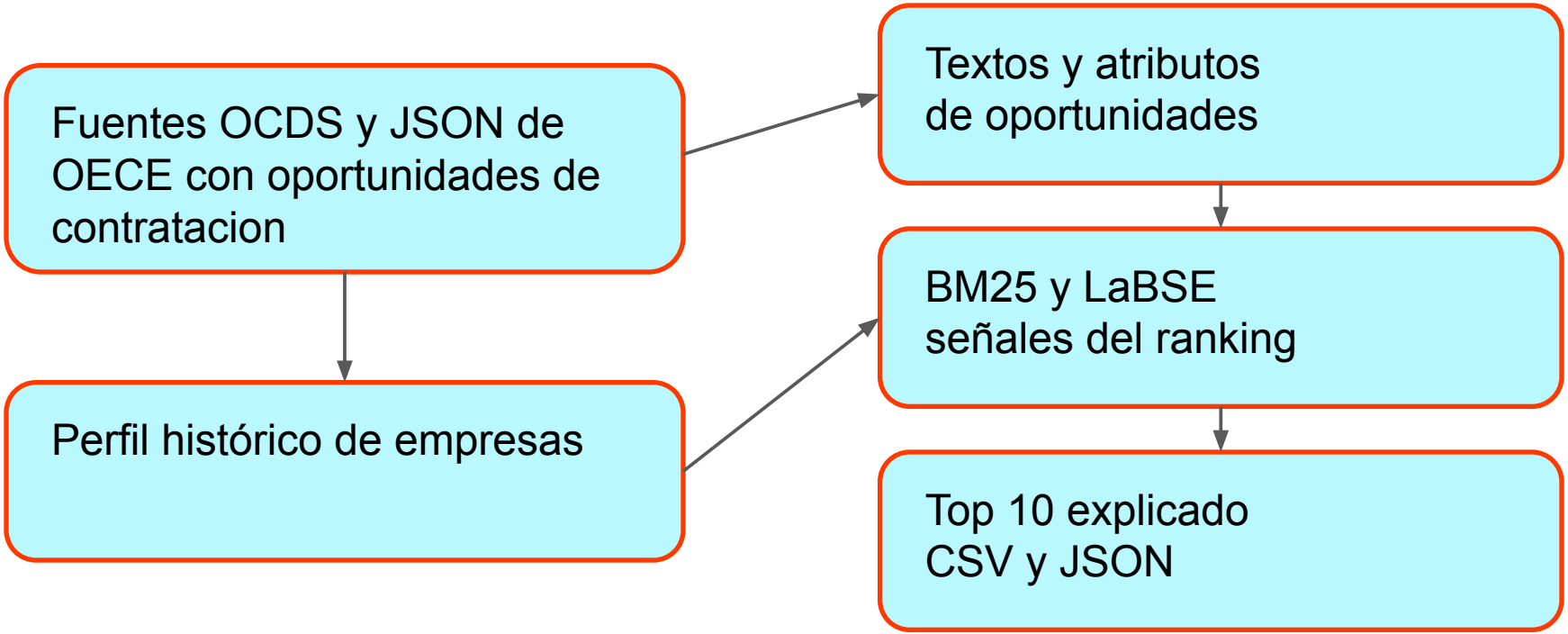
Sebastian Alvarez

Implementación por etapas

- Etapa 1. Variables, fuente y protocolo
- Etapa 2. Corpus y perfiles
- Etapa 3. Procesamiento NLP y LaBSE
- Etapa 4. Ranking híbrido y explicaciones
- Etapa 5. Evaluación y reproducción


1. Variables y protocolo de evaluación
2. Corpus y perfiles empresariales
3. Procesamiento textual y representación
4. Ranking híbrido y explicaciones
5. Evaluación y análisis de resultados
Delimitar unidades de análisis
Definir variables observables
Establecer separación temporal
Fijar condiciones de evaluación
Reunir y preservar fuentes
Integrar versiones y delimitar servicios
Seleccionar perfiles y oportunidades
Construir perfiles históricos
Preparar textos y atributos
Obtener representaciones léxicas y semánticas
Representar textos y antecedentes
Evaluar extracción de campos
Calcular y preparar señales textuales
Incorporar afinidades estructuradas
Integrar configuraciones de recuperación
Generar recomendaciones explicables
Aplicar condiciones comunes
Medir recuperación
Interpretar etiquetas
Analizar errores y validez
Documentar reproducción y alcance

Arquitectura de la solución



Etapa 1. Origen de datos y variables

OECE publica registros OCDS de licitaciones y adjudicaciones

EMPRESA: varias licitaciones anteriores

Agrupo antecedentes por empresa.

Servicios y categorías adjudicados.

Regiones y montos históricos.

OPORTUNIDAD: una licitación publicada

Objeto y descripción del servicio.

Categoría, región y monto publicado.

Fecha y publicación inicial verificable.


```
"tender": {
    "id": "1004592",
    "title": "AS-SM-3-2024-MIDIS-PNAEQW-CS-1",
    "description": "SERVICIO DE SEGURIDAD Y VIGILANCIA PRIVADA PARA LAS INSTALACIONES
    "procuringEntity": {
        "id": "PE-CONSUCODE-200493",
        "name": "PROGRAMA NACIONAL DE ALIMENTACION ESCOLAR QALI WARMA"
    },
    "datePublished": "2024-04-12T10:55:00-05:00",
    "procurementMethod": "open",
    "procurementMethodDetails": "Adjudicación Simplificada",
    "mainProcurementCategory": "services",
    "additionalProcurementCategories": [
        "services"
    ],
    "value": {
        "amount": 0.0,
        "currency": "PEN",
        "currencyName": "Soles",
        "amount_PEN": 0.0
    },
```

Etapa 1. Protocolo y preparación

Reglas del estudio aplicadas sobre los datos descargados

SELECCIÓN Y LIMPIEZA

Seleccionar servicios con texto utilizable.
Integrar publicaciones de la misma licitación.
Revisar fechas y datos faltantes.
Normalizar formatos y limpiar texto.

SEPARACIÓN TEMPORAL

Perfil: antecedentes anteriores a 2025.
Oportunidades: colección anual de 2025.
Adjudicaciones de 2025: solo evaluación.
Evitar información posterior al corte.

CONTROL DE FECHA EN PYTHON LOCAL

release.get("datePublished") lee el campo del JSON descargado.
Interpreto la fecha y la comparo con el corte.
Sin fecha válida, la versión no acredita el corte temporal.

Etapa 2. Corpus de oportunidades

Fuentes originales y versiones identificadas

48 de OECE de 2022 a 2025 (SEACE V3, formato OCDS).
Registros de licitaciones con objeto, ítems, fechas, montos y categorías.
Incluyen entidades compradoras, adjudicaciones y sus proveedores.
Con las adjudicaciones anteriores a 2025 construyo los perfiles empresariales.

34 485 licitaciones de servicios asociadas inicialmente a 2025 por su fecha registrada.
Conservo 33 584 tras verificar su publicación inicial.
Excluyo 901 cuya primera versión verificable se publicó en 2026.
Cada modelo ordena estas oportunidades según su compatibilidad con la empresa.

OCID identifica la licitación. El identificador de versión distingue sus cambios.
Cada archivo conserva fuente, fecha y hash SHA-256.

Etapa 2. Perfiles empresariales

Construyo los perfiles con las adjudicaciones publicadas por OECE

INFORMACIÓN QUE VIENE DE OECE

awards: adjudicación con proveedor, fecha y monto con su moneda.
tender: objeto, descripción, ítems y categoría del servicio.
parties: región de la entidad compradora.

INFORMACIÓN QUE CONTRUYO

Verifico y uno esos datos en historical_awards.csv, una fila por antecedente.
read_awards() lee el CSV. build_profiles() agrupa los antecedentes por empresa.
Resultado: 49 perfiles con 210 antecedentes, al menos dos por empresa.
Una empresa quedó fuera por historial insuficiente, sin reemplazo.

```
for award in awards:
    if award.award_date < cutoff_date:
        grouped[award.supplier_id].append(award)
```

El código toma los antecedentes anteriores a 2025 y reúne los de la misma empresa.

Etapa 3. NLP, LaBSE y embeddings

Representación semántica de los antecedentes y las licitaciones

NLP: PREPARACIÓN DE LOS TEXTOS EN PYTHON
Uno el título, la descripción y el texto técnico de cada servicio.
Tokenizo y divido los textos largos con el tokenizador de LaBSE.
LaBSE: MODELO MULTILINGÜE PREENTRENADO
Sentence Transformers carga LaBSE y encode() genera los vectores.
Transformers aporta el tokenizador. PyTorch ejecuta el modelo.
EMBEDDINGS: REPRESENTACIÓN NUMÉRICA DEL TEXTO
Cada fragmento produce un vector de 768 números.
Promedio los vectores y aplico normalización L2 para compararlos.

Armando el contexto
Preparo los textos para compararlos mediante BM25 y LaBSE. Conservo categoría, región y monto como atributos separados, sin convertirlos en embeddings. En la siguiente etapa combino la comparación textual con esos atributos para ordenar las licitaciones.

Etapa 3. NLP y embeddings LaBSE

Los vectores permiten comparar el significado de los servicios

ENTRADA: TEXTOS DEL CORPUS Y DE LOS PERFILES

Empresa: texto de cada antecedente histórico del perfil.

Licitación: título, descripción y texto técnico disponible.

NLP Y REPRESENTACIÓN SEMÁNTICA

Uno los campos textuales y divido los textos largos en fragmentos.

LaBSE preentrenado convierte cada fragmento en un vector de 768 números.

Promedio y normalizo los vectores de los fragmentos de cada texto.

Para la empresa, promedio y normalizo los vectores de sus antecedentes.

Guardo los vectores para comparar empresa y licitación en la etapa 4.

```
vectors = self._model.encode(
    texts, batch_size=self.batch_size,
    normalize_embeddings=True)
```

encode() recibe textos y devuelve vectores. Sentence Transformers ejecuta LaBSE.

Etapa 4. Ranking híbrido

Comparo cada licitación con el perfil de la empresa

COMPARACIÓN ENTRE EMPRESA Y LICITACIÓN

BM25 compara las palabras del historial con las de la licitación.
LaBSE compara el vector de la empresa con el de cada licitación.
Categoría: coincidencia con los servicios adjudicados anteriormente.
Región: coincidencia con regiones de las entidades del historial.
Monto: cercanía a la mediana histórica en la misma moneda.
Normalizo BM25 y LaBSE entre 0 y 1 para combinarlos.
Multiplico cada señal por su peso y sumo los aportes.
Ordeno por puntaje y devuelvo las diez primeras licitaciones.

$$C = 0{,}35\,L + 0{,}50\,S + 0{,}08\,G + 0{,}04\,R + 0{,}03\,M$$

L: BM25, S: LaBSE, G: categoría, R: región, M: monto.

```
contributions = {name: weights.get(name, 0.0) * values[index]
                 for name, values in components.items()}
score = sum(contributions.values())
```

Etapa 4. Una recomendación explicada

Perfil de seguridad con 25 antecedentes auditados

Aportes al puntaje

Señal	Aporte
BM25	0,348104
LaBSE	0,479428
Categoría	0,080000
Región	0,040000
Monto	0,000000
Total	0,947532

Cómo leer el resultado

En una Licitación de seguridad y vigilancia
Programa Nacional de Centros Juveniles
LaBSE aporta más al puntaje.
Categoría y región coinciden (peso igual al aporte).
Monto inicial: cero, sin aporte.
Total: 0,947532 para ordenar.
Se relaciona con servicios que la empresa ganó anteriormente. Su categoría y la región de la entidad compradora coinciden con su historial. BM25 encuentra coincidencias de palabras y LaBSE similitud de significado. LaBSE suma más puntos por su resultado y el mayor peso asignado. El monto no aporta porque la fuente inicial registra cero. La suma permite ordenar esta licitación entre las recomendaciones.

Etapa 5. Evaluación de recuperación

Contrasto el ranking con adjudicaciones históricas conocidas

REFERENCIA PARA EVALUAR CADA EMPRESA

Un positivo es una licitación adjudicada a esa empresa en 2025.
Comparo sus positivos con las primeras 5 y 10 recomendaciones.
Precision@k: proporción del top-k que coincide con positivos conocidos.
Recall@k: proporción de sus positivos que aparece en el top-k.
F1@k combina precisión y recuperación.
Calculo las métricas por empresa y luego las promedio.
28 pares positivos corresponden a 13 de los 49 perfiles.
Los otros 36 reciben ranking y se reportan sin Recall evaluable.

```
hits = len(set(ranking[:k]) & positives)
precision, recall = hits / k, hits / len(positives)
```

hits cuenta las coincidencias entre el top-k y los positivos de esa empresa.

Etapa 5. Comparación ejecutada

Mismos perfiles y licitaciones, cambio las señales del ranking

Modelo	Precision@10	Recall@10	F1@10
BM25	0,076923	0,495726	0,127158
LaBSE	0,030769	0,230769	0,053613
BM25 + LaBSE	0,084615	0,465812	0,134089
Híbrido con atributos	0,076923	0,427350	0,121269

BM25 recupera más positivos conocidos según Recall@10.
BM25 + LaBSE lidera Precision@10 y F1@10 en esta muestra.
Los atributos no mejoran el top 10 con los pesos fijados.

Etapa 5. Evaluación de los datos extraídos

Verifico los campos del JSON y reviso experiencia por separado

Campos del JSON

50 licitaciones, cuatro campos:
objeto, categoría, región y monto.
Un verificador separado lee el JSON
inicial y aplica las reglas fijadas.
200 valores coinciden con la fuente.
F1 automático: 1,00 por campo.
Mide concordancia con el registro,
con sus límites de información.

Requisitos en PDF

Experiencia: requisitos del postor
en las bases de las licitaciones.
Muestra documental de 50 casos.
27 PDF completos, 25 relecturas.
Faltan los casos 1 y 11.
Luego concilio discrepancias
y fijo la referencia para evaluar.
F1 de experiencia aún no estimable.

Etapa 5. Verificación y demostración

Compruebo puntajes, orden y explicaciones con la evidencia guardada

REPRODUCCIÓN EN PYTHON LOCAL

Leo las señales, los pesos y las fuentes guardadas.

La demo recalcula un ranking y lo compara con la salida archivada.

Exporta recomendaciones con sus aportes y fuentes.

```
.venv/bin/python \
  scripts/demonstrate_annual_recommendations.py \
  --output output/demo_exposicion_nueva
```

196 rankings: 49 empresas por cuatro variantes.

6 582 464 filas y 1 960 explicaciones verificadas.

Última suite completa: 837 pruebas y 8 subpruebas.

Estas pruebas comprueban el software, no su superioridad.

Conclusiones y próximos pasos

La recomendación está implementada y sus límites estan documentados

IMPLEMENTACIÓN

Corpus temporal y perfiles históricos, con rankings explicados.
Conservo los datos, los vectores y el código para reproducirlos.

RESULTADO DEL CONTRASTE

La combinación textual lidera F1@10 en esta muestra.
Agregar atributos requiere más validación antes de afirmar una mejora.

PENDIENTE

Cerrar las dos relecturas y conciliar la referencia documental de experiencia.
Completar su evaluación y discutir el alcance de la hipótesis.

Muchas gracias

Apoyo. Cálculo BM25

Frecuencia de términos, IDF y longitud documental

```
idf = math.log(
    1 + (n_docs - df + 0.5) / (df + 0.5)
)
norm = frequency + self.k1 * (
    1 - self.b + self.b * doc_len / (self.avgdl or 1)
)
score += idf * frequency * (self.k1 + 1) / norm
```

bm25.py, expresiones de BM25.score() reformateadas

k1 = 1,5 y b = 0,75.

Tokenización léxica: minúsculas y normalización de tildes.

Las estadísticas usan la colección anual común.

Apoyo. Agregación semántica alternativa

El máximo por antecedente sigue como experimento separado

Configuración	Recall@10	F1@10
BM25 original	0,495726	0,127158
Combinación original	0,465812	0,134089
Combinación con máximo	0,534188	0,139978
Híbrido original	0,427350	0,121269
Híbrido con máximo	0,495726	0,127158

Reutiliza vectores y pesos. La alternativa no cambia el modelo predeterminado.

El intervalo del contraste principal incluye cero.

Código de apoyo. Texto del JSON

Leo los ítems de la licitación para construir su texto técnico

```
parts: list[str] = []
for item in tender.get("items", []):
    parts.append(item.get("description", ""))
    parts.append(item.get("classification", {}).get("description", ""))
    parts.extend(
        value.get("description", "")
        for value in item.get("additionalClassifications", [])
    )
return " ".join(part for part in parts if part).strip()
```

Entrada: tender, el bloque de la licitación en el JSON.
El for recorre los ítems. get() lee sus descripciones y clasificaciones.
join() une los textos disponibles y strip() limpia los extremos.
Ese texto complementa el título y la descripción del servicio.

Código de apoyo. Perfiles empresariales

Agrupo adjudicaciones anteriores al corte por identificador de empresa

```
grouped: dict[str, list[Award]] = defaultdict(list)
for award in awards:
    if award.award_date < cutoff_date:
        grouped[award.supplier_id].append(award)
return {
    supplier_id: CompanyProfile(
        supplier_id, cutoff_date, sorted(items, key=lambda x: x.award_date)
    )
    for supplier_id, items in grouped.items()
    if len(items) >= minimum_awards
}
```

Entrada: adjudicaciones auditadas, leídas desde historical_awards.csv.
El if conserva antecedentes anteriores a 2025. append() los agrupa.
CompanyProfile reúne el historial de cada empresa y ordena sus fechas.
minimum_awards exige al menos dos antecedentes verificables.

Código de apoyo. Generación de embeddings

Sentence Transformers ejecuta LaBSE con los textos que preparo

```
self._model = SentenceTransformer(self.model_name)

vectors = self._model.encode(
    texts,
    batch_size=self.batch_size,
    normalize_embeddings=True,
    show_progress_bar=len(texts) > self.batch_size,
)
return vectors.tolist()
```

SentenceTransformer carga LaBSE preentrenado para usarlo localmente.
texts contiene los textos o fragmentos. encode() produce sus vectores.
batch_size permite ejecutarlos por lotes y controlar el uso de memoria.
normalize_embeddings aplica L2. tolist() devuelve listas de números.

Código de apoyo. Normalización de puntajes

Llevo los resultados de BM25 y LaBSE a una escala común

```
def minmax(values: list[float]) -> list[float]:
    if not values:
        return []
    low, high = min(values), max(values)
    if math.isclose(low, high):
        return [1.0 if high > 0 else 0.0 for _ in values]
    return [(value - low) / (high - low) for value in values]
```

values contiene los puntajes de un método para todas las licitaciones.
min() y max() encuentran los extremos para esa empresa.
La última línea sitúa cada puntaje entre 0 y 1, antes de aplicar los pesos.
Un valor alto expresa posición relativa en el corpus, no un porcentaje.

Código de apoyo. Categoría y región

Comparo los atributos con el historial, sin generar embeddings

```
candidate_category = opportunity.category.strip().lower()
informative_categories = profile.categories - GENERIC_CATEGORIES
category = float(
    bool(candidate_category)
    and candidate_category not in GENERIC_CATEGORIES
    and candidate_category in informative_categories
)
region = float(
    bool(opportunity.region.strip())
    and opportunity.region.strip().lower() in profile.regions
)
```

strip() y lower() uniformizan el texto de los atributos.
category vale 1 si coincide con una categoría informativa del historial.
region vale 1 si coincide con una región de las entidades compradoras.
Al ponderar, una coincidencia completa aporta 0,08 o 0,04.

Código de apoyo. Comparación del monto

Mido cercanía a la mediana histórica en la misma moneda

```
amount = 0.0
median = profile.median_amount_in(opportunity.currency)
if (
    median and opportunity.amount is not None
    and math.isfinite(opportunity.amount)
    and opportunity.amount > 0
):
    amount = math.exp(
        -abs(math.log(opportunity.amount) - math.log(median))
    )
```

median usa montos históricos positivos en la moneda de la licitación.
El if exige un monto disponible, finito y mayor que cero.
La fórmula da 1 cuando los montos son iguales y baja al separarse.
Si la fuente registra cero, el aporte monetario se mantiene en cero.

Código de apoyo. Puntaje y orden del ranking

Cada recomendación conserva cuánto aportó cada componente

```
contributions = {
    name: weights.get(name, 0.0) * values[index]
    for name, values in components.items()
}
score = sum(contributions.values())
dominant = max(contributions, key=contributions.get)

ranked.sort(key=lambda item: (-item.score, item.opportunity_id))
return ranked if top_k is None else ranked[:top_k]
```

values[index] toma la señal de esta licitación y la multiplica por su peso.
sum() obtiene el total. dominant identifica el mayor aporte ponderado.
sort() ordena de mayor a menor y desempata por identificador de licitación.
ranked[:top_k] devuelve las primeras recomendaciones.

Código de apoyo. Métricas por empresa

Cuento las adjudicaciones conocidas que aparecen en el top-k

```
for k in (5, 10):
    hits = len(set(ranking[:k]) & positives)
    precision, recall = hits / k, hits / len(positives)
    result[f"precision@{k}"] = precision
    result[f"recall@{k}"] = recall
    result[f"f1@{k}"] = (
        2 * precision * recall / (precision + recall)
        if precision + recall else 0.0
    )
```

ranking contiene identificadores ordenados. positives es la referencia.
La intersección & encuentra positivos conocidos dentro de las primeras k.
Precision divide aciertos entre k. Recall, entre los positivos de la empresa.
F1 combina ambos. Calculo por empresa y después promedio.